ENSEMBLE

GenAI & Agentic AI

Candidate Assessment

Objective

Design and build an **agentic AI application** that demonstrates your ability to work with large language models, tool integration, prompt engineering, and autonomous multi-step reasoning. The focus is on **system design, code quality, and practical GenAI engineering skills** — not on achieving perfect outputs.

Details

Stack — Use any LLM provider (OpenAI, Anthropic, open-source, etc.). Include setup instructions and any required API key placeholders in your README. You may use any agentic framework (LangChain, LangGraph, CrewAI, AutoGen, or a custom implementation).

Domain — Choose one of the following domains for your agent, or propose your own:

- Healthcare Q&A agent that retrieves and summarizes information from PubMed or clinical guidelines.
- Data Analysis agent that accepts a CSV, asks clarifying questions, generates analysis code, executes it, and returns insights.
- Document Processing agent that extracts structured data from unstructured documents (invoices, medical records, contracts).
- DevOps/SRE assistant that triages alerts, queries logs, and suggests root cause with supporting evidence.

Tasks

1. Agent Architecture & Design

- Define the agent's goal, available tools, and decision-making flow.
- Create an architecture diagram (can be a simple Mermaid/ASCII diagram in the README).
- Implement the agent loop: reason → plan → act → observe → respond.
- Handle at least one tool/function call (e.g., web search, database query, API call, code execution).

2. Prompt Engineering & LLM Integration

ENSEMBLE

- Write well-structured system prompts and few-shot examples where appropriate.
- Implement structured output parsing (e.g., JSON mode, tool-call schemas, Pydantic models).
- Demonstrate at least one advanced technique: chain-of-thought prompting, self-reflection / self-critique, retrieval-augmented generation (RAG), or multi-agent collaboration.

3. Robustness & Error Handling

- Handle LLM API failures gracefully (retries, timeouts, fallback responses).
- Implement input validation and guard-rails (e.g., content filtering, token-limit checks).
- Add logging for each agent step so the full reasoning trace is inspectable.

4. Evaluation & Observability

- Define 3–5 representative test scenarios and expected outcomes.
- Provide a lightweight evaluation: compare agent outputs against expected results (qualitative or quantitative).
- Include a trace/log of at least one full agent run showing the reasoning chain, tool calls, and final output.

Expectations

Code Quality

- Code must be modularized into functions, classes, or well-separated modules (not a single monolithic script or notebook).
- Agent logic, tool definitions, prompts, and evaluation should be logically separated.
- Configuration (model name, temperature, API keys) should be externalized (env vars, config file), not hard-coded.

Documentation

- A README that explains: the chosen domain and agent design rationale, architecture overview (diagram preferred), how to set up and run the project, how to run the evaluation/test scenarios.

What We Value

This assessment is designed to evaluate practical GenAI engineering skills. We prioritize:

Thoughtful Design	Clear rationale for agent architecture, tool selection, and prompt design.
Working Software	The application runs end-to-end.

ENSEMBLE™

Prompt Craft	Effective system prompts, structured outputs, and context management.
Error Awareness	Graceful handling of LLM quirks: hallucinations, refusals, token limits, latency.
Clean Code	Readable, modular, and well-documented. Follows Python best practices.

Deliverables

Provide a GitHub repository containing:

Code

- Python project with modular structure.
- A requirements.txt file.
- A .env.example file with placeholder API keys.
- README file with setup instructions and high-level description.

Include the exact command(s) to reproduce. Example:

```
pip install -r requirements.txt
cp .env.example .env   # add your API key
python src/agent.py --domain healthcare --query "What are the latest treatment
options for Type 2 diabetes?"
python src/evaluate.py --scenarios tests/scenarios.json
```

Agent Run Report

- A Word document, PDF, or Markdown file.
- Include: architecture diagram, sample agent traces (reasoning chain + tool calls + final output), evaluation results across test scenarios, and a short written summary of design decisions and trade-offs.
- Formatting/polish is not important — focus on clarity of explanation.

Bonus (Optional)

These are not required but will strengthen your submission:

- Multi-agent orchestration (e.g., a planner agent that delegates to specialist agents).
- Memory / conversation persistence across sessions.
- Streaming responses for real-time user interaction.
- Containerized deployment (Dockerfile or docker-compose).
- A simple UI (Streamlit, Gradio, or a web frontend).